

动态规划

2019寒假



动态规划基本知识

- ◆ 用于解决多阶段决策性最优解的问题
- ◆ 先决条件：无后效性和最优子结构
- ◆ 实现方法：递推和记忆化搜索
- ◆ 高效原因：重复子问题不重复处理
- ◆ 解题步骤：划分阶段—确定状态—转移方程---寻求边界和特殊情况---代码实现

动态规划模型分类

- ◆ 线性动态规划（前后线性关系）
- ◆ 坐标动态规划
- ◆ 区间动态规划（区间范围内处理）
- ◆ 树形动态规划（基于树上关系）
- ◆ 背包动态规划（元素离散关系）
- ◆ 状压动态规划（状态压缩的关系）
- ◆ 数位动态规划（对数字按位处理）
- ◆ 动态规划优化（主要对于时间和空间的优化）

线性类动态规划

- ◆ 基本思想： $dp[i]$ 表示到 i 这个阶段的最优解
- ◆ 动态规划用维数来维护更多的状态和特征。
- ◆ $dp[i][j], dp[i][j][k], \dots$

上升子序列与不下降子序列

最长上升子(不下降)序列

- ◆ 设有整数序列 $b_1, b_2, b_3, \dots, b_m$ ，若存在下标 $i_1 < i_2 < i_3 < \dots < i_n$ ，且 $b_{i_1} < b_{i_2} < b_{i_3} < \dots < b_{i_n}$ ，则称 $b_1, b_2, b_3, \dots, b_m$ 中有长度为 n 的不下降序列 $b_{i_1}, b_{i_2}, b_{i_3}, \dots, b_{i_n}$ 。
- ◆ 求序列 $b_1, b_2, b_3, \dots, b_m$ 中所有长度(n)最大不下降子序列
- ◆ 输入：整数序列。
- ◆ 输出：最大长度 n 和所有长度为 n 的序列个数。

上升子序列与不下降子序列

基本求法：

1、设 $dp[i][j]$ 前 i 个数字以第 j 个数结尾的最大上升子序列长度

则： $dp[i][j] = \max \{dp[i-1][k] + 1\} \quad (1 \leq k < j \leq m, b_j < b_i)$;

边界为 $dp[1][1] = 1$; 答案即为 $ans = \max(dp[i][k])$

由于在求 $dp[i][k]$ 时，除了 $dp[i][i]$ 需要从新求以外，其他

$dp[i][k]$ 与 $dp[i-1][k]$ 答案相同，因此可以优化维数。

2、设 $dp[i]$ 为前 i 个数中以第 i 个数结尾的最大上升子序列长度，

则 $dp[i] = \max \{dp[j] + 1\} \quad (1 \leq j < i \leq m, b_j < b_i)$; 边界为 $dp[1] = 1$

上升子序列与不下降子序列

3、设 $t[i]$ 为前 i 个数中以第 i 个数结尾的最长上升子序列的个数, 则

$$t[i] = \sum t[j] \quad (1 \leq j < i \leq m, b_j < b_i, dp[i] = dp[j] + 1)$$

初始为 $t[i]=1$; 当 $dp[i]=$ 最大长度时, 将 $t[i]$ 累加

举例:

	1	2	3	4	6	5	8	10	9
dp:	1	2	3	4	5	5	6	7	7
t:	1	1	1	1	1	1	2	2	2

答案: $dp=7$ 时, 边界为 $\sum t=4$

```
dp[1]=t[1]=1;
for(int i=2;i<=n;i++){
    for(int j=1;j<i;j++){
        if(a[i]>a[j]){
            if(dp[i]<dp[j]+1){
                dp[i]=dp[j]+1;t[i]=t[j];
            } else if(dp[i]==dp[j]+1){
                t[i]+=t[j];
            }
        }
    }
}
```

优化: 由于以第 i 个数结尾时找的是由前 $i-1$ 个数中比第 i 个数小, 且长度最长的那个转移过来。那么这个寻找过程是否可以优化呢?

上升子序列与不下降子序列(优化)

- ◆ 由于以第 i 个数结尾时找的是由前 $i-1$ 个数中比第 i 个数小，且长度最长的那个转移过来。那么这个寻找过程是否可以优化呢？
- ◆ 初看是没办法优化的，但进一步思考分析，对于相同长度的上升子序列，我们肯定是希望序列末尾元素值越小越好，这样后面便可以加更多的元素，如果我们维护一个最优的序列呢？则对于第 i 个数是否有更快的处理方法？

上升子序列与不下降子序列(优化)

- ◆ 如果我们维护出一个相同长度，末尾元素最优的序列，这发现这个序列长度成单调性(长度依次增加啊),且序列元素的值也成单调性(随着长度增加，值肯定也是越来越大)。
- ◆ 因此，对于第*i*个元素我们便和序列末尾元素比较，如果比它大直接加入(说明构成一个更长的序列)；如果比它小，则由第*i*个元素去更新序列第*i*个数结尾中长度最长的最有序列(即长度最长以第*i*个数结尾的末尾元素最优)：如：

给定序列: 1 2 3 4 6 5 8 10 9

维护序列: 1 2 3 4 5 8 9

- 基于此，我们便可以二分动态维护这个序列，最后维护序列的长度就是我们最长上升子序列的长度。时间复杂度 $n \cdot \log(n)$
- 思考：维护序列是不一定是一个合法的最长上升子序列？

上升子序列与不下降子序列(优化)

- ◆ 基于此，我们便可以二分动态维护这个序列，最后维护序列的长度就是我们最长上升子序列的长度。时间复杂度 $n \cdot \log(n)$
- ◆ 二分STL函数：lower_bound和upper_bound
- ◆ lower_bound:求大于等于一个数的第一个数的位置。
- ◆ upper_bound:求大于一个数的第一个数的位置。
- ◆ 在上升子序列中用: upper_bound(因为要求严格递增)
- ◆ 在不下降子序列中用lower_bound
- ◆ 二个函数常见用法：
https://blog.csdn.net/qq_40160605/article/details/80150252

上升子序列与不下降子序列(转化)

◆ Hdu 5256

- ◆ 题意：给出一个数列， $a_1, a_2, a_3, a_4, \dots, a_n$ ，现在需要修改尽量少的元素，使得这个序列严格递增。
- ◆ 问题分析：是否就是对原序列求一个最长上升子序列长度，然后用总长度-最长上升子序列长度呢？
- ◆ 答案是否定的，如：1 2 4 3 4 5这个序列的最长上升子序列长度为5，则修改最少元素为1个，明显是错误的。这里显然需要修改4个。那这里怎么做呢？

上升子序列与不下降子序列(转化)

- ◆ 问题分析：透过表面看本质
- ◆ 因为 $a[i] < a[i+1]$
- ◆ 又因为 $a[i], a[i+1]$ 均为整数
- ◆ 所以 $a[i+1] - 1 \geq a[i]$
- ◆ 变形得 $a[i+1] - (i+1) \geq a[i] - i$
- ◆ 令 $b[i] = a[i] - i$;
- ◆ 所以可以求出 $b[i]$ 的最长不下降子序列。
- ◆ 再用 $n - \text{len}$ 即为需要改变的最少个数

线性类动态规划

◆ 护卫队(P1594)

- ◆ 护卫车队在一条单行的街道前排成一队，前面河上是一座单行的桥。因为街道是一条单行道，所以任何车辆都不能超车。桥能**承受一个给定的最大承载量**。为了控制桥上的交通，桥两边各站一个指挥员。护卫车队被分成几个组，**每组中的车辆都能同时通过该桥**。当一组车队到达了桥的另一端，该端的指挥员就用电话通知另一端的指挥员，这样**下一组车队才能开始通过该桥**。**每辆车的重量是已知的，任何一组车队的重量之和不能超过桥的最大承重量**。被分在同一组的每一辆车都以其最快的速度通过该桥。**一组车队通过该桥的时间是用该车队中速度最慢的车通过该桥所需的时间来表示的**。问题要求计算出全部护卫车队通过该桥所需的最短时间值。

护卫队

- ◆ 问题分析：题面较长，其实质想表达的是有一个长为 n 的车队，要在车辆顺序不变（不能超车）的情况下把它分为若干个小车队过桥，桥上只能有一个车队，抽象地讲就是把一个数列分为若干份。问怎么分使得代价最优。
- ◆ 因此我们完全可以考虑用动态规划来解，动态规划的目的就是来找到分开小车队的断点。
- ◆ 所以我们用 $dp[i]$ 表示前 i 辆车全部通过的最短时间，则 $dp[i] = \min(dp[i], dp[j-1] + st[j][i])$, $1 \leq j \leq i$
- ◆ $st[j][i]$ 表示 j 到 i 组成一队，通过桥花费的最小时间。
- ◆ DP 方程表达的含义就是以 j 为断点，前 $j-1$ 辆车通过的最优时间 + $[j]$ 与 $[i]$ 合成一队花费的时间。

护卫队(参考代码)

```
int x,y;
scanf("%d%d",&maxv,&len,&n);
for(int i=1;i<=n;i++){
    scanf("%d",&x);
    w[i]=w[i-1]+x;
    s[i]=len*60.00000/y;
}
dp[0]=0;
for (int i=1;i<=n;i++) dp[i]=999999999999999.0;
for(int i=1;i<=n;i++){
    double mint=-999999999999999.0;//mint表示车
    for(int j=i;j>=1;j--){
        if(w[i]-w[j-1]>maxv) break;//后面一队超
        else {
            mint=max(mint,s[j]);
            dp[i]=min(dp[i],dp[j-1]+mint);//f[i]
        }
    }
}
printf("%.11f\n",dp[n]);
```

Print Article(HDU 3507)

- ◆ 题目大意:
- ◆ 给出N个单词，每个单词有个非负权值 C_i ，现要将它们分成连续的若干段，每段的代价为此段单词的权值和的平方，还要加一个常数M，即 $(\sum C_i)^2 + M$ 。现在想求出一种最优方案，使得总费用之和最小。

问题分析

- ◆ 根据护卫队同样的思路，容易写出方程：
$$dp[i] = \min\{dp[j] + (s[i] - s[j])^2 + M\} (0 \leq j \leq i-1)$$
- ◆ 其中s是前缀和
- ◆ $dp[i]$ 表示输出到i的时候最少的花费。
可是范围是500000，又不能用单调队列，那怎么办呢？

- ◆ 我们假设在求解 $dp[i]$ 时，存在 $j, k (j > k)$ 使得从 j 转移比从 k 转移更优，那么需要满足条件：

$$dp[j] + (s[i] - s[j])^2 + M < dp[k] + (s[i] - s[k])^2 + M$$

- ◆ 展开上式：移项并消去再合并同类项得（把 j 和 k 放到同一边）

$$dp[j] - dp[k] + s[j]^2 - s[k]^2 < 2s[i](s[j] - s[k])$$

- ◆ 把 $S[j] - S[k]$ 除过去，得到 $\frac{dp[j] - dp[k] + s[j]^2 - s[k]^2}{s[j] - s[k]} < 2s[i]$

- ◆ 我们设 $f[x] = dp[x] + s[x]^2$ ，就化成了 $\frac{f[j] - f[k]}{s[j] - s[k]} < 2s[i]$

即当且仅当 $(j > k)$ 且 $\frac{f[j] - f[k]}{s[j] - s[k]} < 2s[i]$ 时， j 对更新 $dp[i]$ 比 k 更新 $dp[i]$ 优。

休息一下qwq。这个东西好像斜率。

- ◆ 当一个数的 dp 值求完了，它的 f 值也跟着确定，我们就可以在空间中绘制出点 $(S[i], f[i])$ ，这个点代表已经求出 dp 值的一个点。

- 当我们要求解 $dp[t]$ 时，如果可用的集合里存在这样三个点，位置关系如图所示：

那么显然

$$\frac{f[j]-f[k]}{S[j]-S[k]} > \frac{f[i]-f[j]}{S[i]-S[j]}$$

这时候他们和 $2S[t+1]$ 的关系有3种：

$$\frac{f[j]-f[k]}{S[j]-S[k]} > \frac{f[i]-f[j]}{S[i]-S[j]} > 2S[t+1]$$

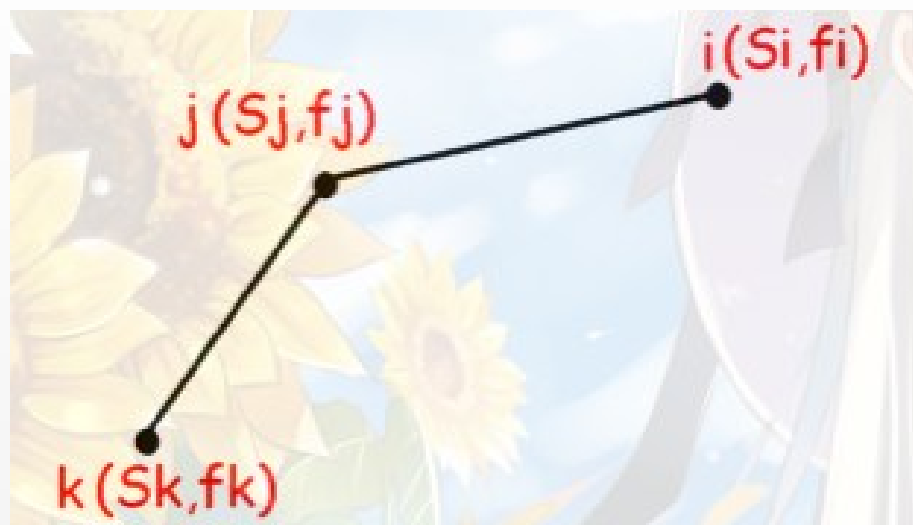
那么 j 比 i 优， k 比 j 优。

$$\frac{f[j]-f[k]}{S[j]-S[k]} > 2S[t+1] > \frac{f[i]-f[j]}{S[i]-S[j]}$$

那么 i 比 j 优， k 比 j 优。

$$2S[t+1] > \frac{f[j]-f[k]}{S[j]-S[k]} > \frac{f[i]-f[j]}{S[i]-S[j]}$$

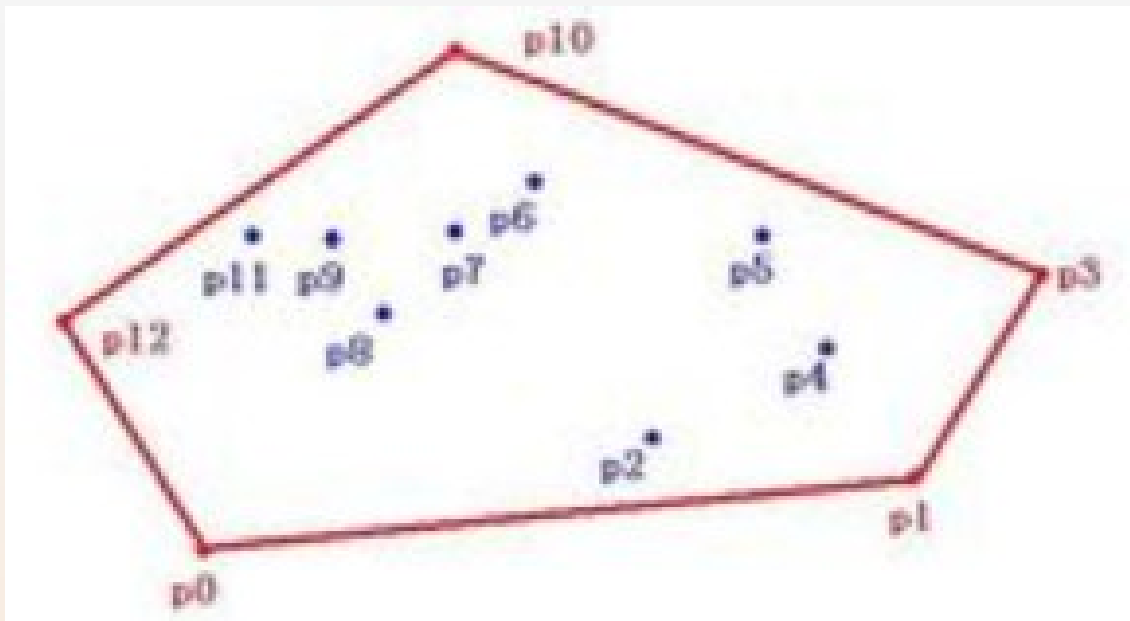
那么 i 比 j 优， j 比 k 优。



综上，不管什么样的 $S[t+1]$ ，从 j 转移都不会是最佳方案。那么用一个数据结构维护一个凸包（下凸），每加入一个点就删去一些点，使其维持凸包的形态。最优转移一定在这个凸包中

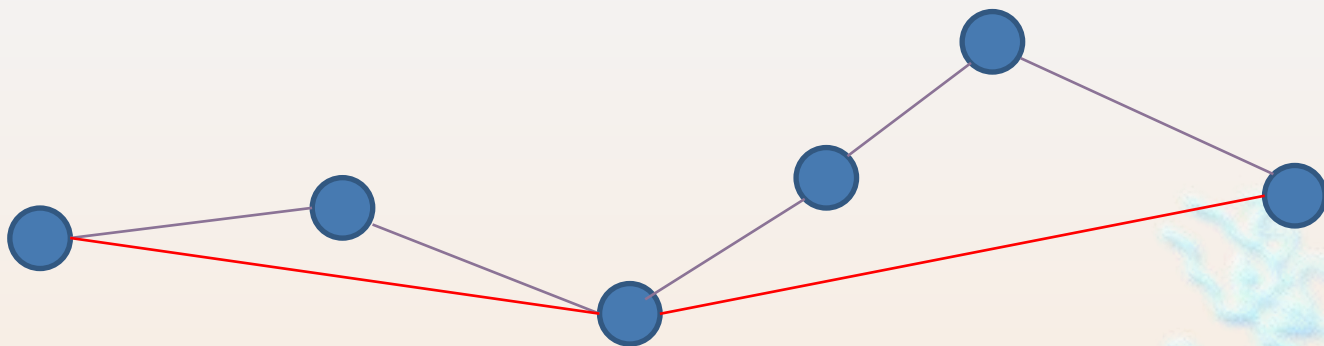
凸包

- ◆ 给定二维平面上的点集，凸包就是将最外层的点连接起来构成的凸多边形，它能包含点集中所有的点。



凸包的求法

- ◆ 把平面上的点，按照x坐标从小到大排序，然后找出离y轴最近和最远的两个点，把凸包分成上凸壳和下凸壳两个部分，然后分别求。
- ◆ 下凸壳的求法：栈模拟



- ◆ 但还是不能 $O(1)$ 。在凸包里，谁又是最最优呢？
- ◆ 首先一定数据结构里的凸包一定会是这样的：



- ◆ 假设 j_i 的斜率 $>2S[t+1]$ 且 k_j 的斜率 $<2S[t+1]$ 从图形特点我们可以发现 j 点比所有比 k 小的点都优，比所有比 i 大的也优。所以对于我们二分查找斜率比 $2S[t+1]$ 小但编号最大的点，就是最优的转移点。由于 $S[i]$ 也满足单调性，我们还可以直接维护一个单调队列就能解决这个问题。
- ◆ 推广一下，如果不等式右侧不是像 $S[i+1]$ 这样的单调函数，二分就好了。

单调队列的维护

- ◆ 第一步：找斜率比 $2s[t+1]$ 小但编号最大的点

```
while(L<R&&slop(q[L],q[L+1])<=sum[i]) L++;
```

- ◆ 第二步：维护保持下凸壳图形的性质

```
while(L<R&&slop(q[R-1],q[R])>slop(q[R],i)) R--;
```

```
double slop(int k,int j)
{
    return (double)(dp[j]+sum[j]*sum[j]-dp[k]-sum[k]*sum[k])/((double)(2*(sum[j]-sum[k])));
}
```

参考博客：<https://www.cnblogs.com/orzzz/p/7885971.html#4050706>

代码实现

```
for(int i=1;i<=n;i++)
{
    scanf("%lld",&sum[i]);
    sum[i]+=sum[i-1];
    if(sum[i]==sum[i-1]) i--,n--;
}
int L=1,R=1;
dp[0]=0;
for(int i=1;i<=n;i++)
{
    while(L<R&&slop(q[L],q[L+1])<=sum[i]) L++;//查找斜率比sum[i]大的第一个位置
    dp[i]=dp[q[L]]+(sum[i]-sum[q[L]])*(sum[i]-sum[q[L]])+k;
    while(L<R&&slop(q[R-1],q[R])>slop(q[R],i)) R--;
    dl[++r]=i;
}
printf("%lld\n",dp[n]);
```


斜率优化dp总结

- ◆ 1、写出dp方程
- ◆ 2、观察dp方程是否中，推出dp[i]的值是否与枚举1—j转移而来，且由于i有关系，形如 $f[i] = \min \text{ or } \max \{f[j] + (x[i] - x[j])^2\}$ ，这样的样式。
- ◆ 3、对dp方程进行变形，得出斜率表达式。
- ◆ 4、用斜率优化dp，代码实现。注意：求最小值是维护下凸壳，求最大值是维护而上凸壳。

Picnic Cows (HDU3045)

◆ 题目大意：

给出一个有 N ($1 \leq N \leq 400000$) 个正数的序列，要求把序列分成若干组（可以打乱顺序），每组的元素个数不能小于 T ($1 < T \leq N$)。每一组的代价是每个元素与最小元素的差之和，总代价是每个组的代价之和，求总代价的最小值。

Picnic Cows

- ◆ **问题分析：**由于可以打乱序列顺序，又知道代价为组内每个元素与最小值差之和，故想到贪心，先将序列排序。排完序后，会发现和护卫队问题就很相似了，依然是把序列分成若干份，然后最后总体代价最优。所以我们依然可以用 $dp[i]$ 表示前 i 个数分成若干份的最优代价，则方程如下：
- ◆ $dp[i] = \min(dp[i], dp[j] + (a[j+1 \sim i] - \text{mink}))$ ($0 \leq j < i$)
- ◆ mink 表示 $j+1 \sim i$ 范围内的最小值，排序后即 $a[j+1]$ 。
- ◆ 上式变形得到： $dp[i] = \min(dp[j] + \text{sum}[i] - \text{sum}[j] - a[j+1] * (i - j))$
- ◆ $\text{sum}[i]$ 表示前 i 个元素之和(前缀和)
- ◆ 这个方程的思路就是枚举 j 不断地计算状态值更新答案。但是数据规模达到了40000，这种以 $O(n^2)$ 为绝对上界方法在这里明显行不通，怎么办？？？

斜率优化

- ◇ 变形得出斜率表达式为：
- ◇ $f[i] = dp[i] - sum[i] + a[i+1] * i$
- ◇ $\frac{f[j] - f[k]}{a[j+1] - a[k+1]} < i$

线性类动态规划

- ◆ 摆渡车(NOIP2018 普及组第三题)
- ◆ 有 n 名同学要乘坐摆渡车从人大附中前往人民大学，第 i 位同学在第 t_i 分钟去等车。只有一辆摆渡车在工作，但摆渡车容量可以视为无限大。摆渡车从人大附中出发、把车上的同学送到人民大学、再回到人大附中（去接其他同学），这样往返一趟总共花费 m 分钟（同学上下车时间忽略不计）。
- ◆ 摆渡车要将所有同学都送到人民大学。凯凯很好奇，如果他能任意安排摆渡车出发的时间，那么这些同学的等车时间之和最小为多少呢？注意：摆渡车回到人大附中后可以即刻出发。

练习题目(落谷)

◆ P4072

◆ P4360

◆ P2365

◆ CF311B

◆ 作业博客:

<https://www.cnblogs.com/Judge/p/9551035.html>



背包类动态规划

- ◆ 详见背包九讲
- ◆ 背包类问题的核心：01背包，物品选还是不选，至于怎么选？选多少件？有什么依赖关系等均可在01背包上变化维护。
- ◆ 01背包核心： $dp[v]$ 表示构造容量为 v 的最大价值。
- ◆ $dp[v]=\max(dp[v], dp[v-a[i]])$ ，选和不选。

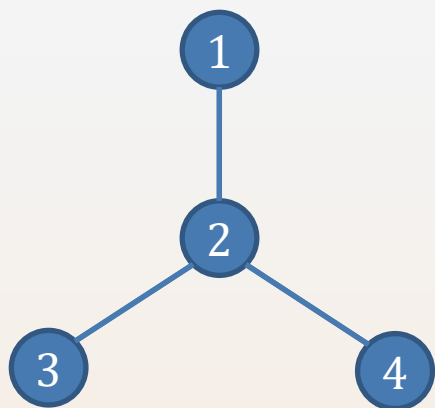
树形动态规划

- ◆ 树形DP是基于树上的动态规划，一般采用记忆化搜索的方法实现，用于统计树上的最优解问题。
- ◆ 树上dp一般我们都是一步一步统计更新，但是不排除类似于树上倍增这样子遍历，这种我们把它理解为动态DP。
- ◆ 如NOIP2018 day2 T3便是这个问题。

树形动态规划---树的重心

◆ 树的重心

- ◆ 对于一颗n个结点的无根树，找到一个点，使得把树变成以该点为根的有根树时，最大子树的结点数最小。换句话说，**删除这个点后最大联通块的结点数最小，那么这个点就是树的重心。如下图，该树的重心即为2号结点**



如何求解树的重心呢？

朴素方法：枚举每一个点，求得以这个点为根的最大子树大小，然后取其最小值即可，时间复杂度($n*n$)

优化方法：设 $siz[i]$ 表示以 i 为根的子树的结点数,任选一个结点为根,把无根树转换成有根树,可知 $siz[i] = \sum_{j \in S[i]} (siz[j] + 1)$ 。我们只需要一遍dfs，即可求出 $siz[i]$ ；再分析，在删除结点 i 后，最大的联通块有多少个结点呢？这里最大的联通块只能是两个部分，一个是由 i 点下方的子树和 i 点上方的联通块组成， i 点下方子树中最大的有 $\max(siz[j])$ 个结点， i 的“上方子树”中有 $n - siz[i]$ 个结点。在动态规划的过程中就可以根据定义顺便找出树的重心了。

树形动态规划---树的中心

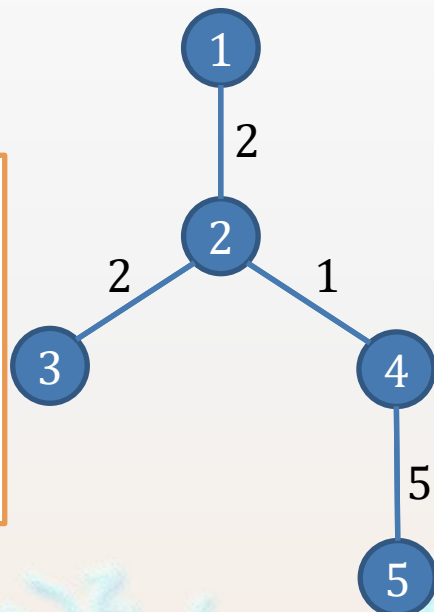
- ◆ **树的中心：**
- ◆ 给出一棵边带权的树，求树中的点，**使得此点到树中的其他节点的最远距离最近**
- ◆ 如右图，4号结点即为树的中心

问题分析： $n*n$ 的算法略，

我们分析：从任意一点 i 出发的最长路径的可能形态有两种。

1 从 i 点出发向上，即终点不在以 i 为根的子树中的最长路径为 $u[i]$

2 从 i 点出发向下，即终点在以 i 为根的子树中的最长路径长度为 $d[i]$



树形动态规划---树的中心

问题分析: $n*n$ 的算法略,

我们分析: 从任意一点 i 出发的最长路径的可能形态有两种。

(1)从 i 点出发向上, 即终点不在以 i 为根的子树中的最长路径为 $u[i]$

(2)从 i 点出发向下, 即终点在以 i 为根的子树中的最长路径长度为 $d[i]$

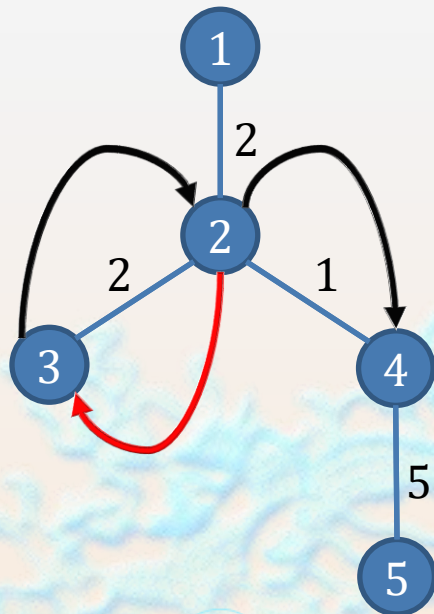
这里关键如何求出 $u[i]$ 。

i 向上的路径必经过 $(i,prt[i])$, 而 i 点的父结点 $prt[i]$ 又引出两条路径, 一条是 $prt[i]$ 向上的最长路径, 其长度为 $u[prt[i]]$;而另一条是 $prt[i]$ 向下的路径,该路径不能途径 i 点, 否则会产生重复计算, 如下图:

因此设:

$d1[i]$ 表示以 i 为根的子树中, i 到叶子结点的距离最大值;
 $d2[i]$ 表示以 i 为根的子树中, i 到叶子结点的距离次大值;
分别用 $c1[i]$ 和 $c2[i]$ 记录 $d1[i]$ 和 $d2[i]$ 是从哪个子树更新而来的。

$u[i]$ 表示除了以 i 为根的子树中的叶子结点外, 其他的叶子结点到 i 的最大值。



树形动态规划---树的中心

算法实现步骤

1. 首先，一遍树形dp算出 $d1, d2, c1, c2$ ，令 j 是 i 的儿子，则：

- ◆ (1) 若 $d1[j] + dis[i][j] > d1[i]$ ，则 $d2[i] = d1[i]$ ； $d1[i] = d1[j] + dis[i][j]$ ；
- ◆ (2) 否则，若 $d1[j] + dis[i][j] > d2[i]$ ，则 $d2[i] = d1[j] + dis[i][j]$ ；

2. 然后，设 $p1[i] = x$ ，

- ◆ 若 $c1[x] \neq i$ 即 $d1[x]$ 不从 i 更新而来，那么
 $u[i] = \max(d1[x], u[x]) + dist[x][i]$ ；
- ◆ 若 $c1[x] = i$ 即 $d1[x]$ 从 i 更新而来，那么 $u[i] = \max(d2[x], u[x]) + dist[x][i]$ 。

3. 最后在 n 个结点中找出最大值，即

- ◆ $t[i] = \max\{u[i], d1[i]\} \quad (1 \leq i \leq n)$

4. 树的中心： $ans = \min\{t[i]\} \quad (1 \leq i \leq n)$

状态压缩动态规划

- ◆ 状态压缩动态规划：是另一类非常典型的动态规划，通常使用在NP问题的小规模求解中（数据规模小，但是状态很多），虽然是指数级别的复杂度，但速度比搜索快，其思想非常值得借鉴，这也是近些年很喜欢考的题目。
- ◆ 状态压缩类题目经典的做法是把复杂较多的单个形用二进制压缩成一个整体进行变换解决，利用位运算的整体处理思想，快速变换。

常见的位运算

- ◆ 程序中的所有数在计算机内存中都是以二进制的形式储存的。
- ◆ 位运算就是直接对整数在内存中的二进制位进行操作(即把整数转换成二进制进行操作)
- ◆ 常见的位运算符: $\&$ $|$ $\wedge$ $\sim$ $\gg$ $\ll$
- ◆ $\&$: 按位与, 如果两个相应的二进制位都为1, 则该位的结果值为1, 否则为0。
 $|$: 按位或, 两个相应的二进制位中只要有一个为1, 该位的结果值为1, 否则为0。
 $\wedge$: 按位异或, 若参加运算的两个二进制位值相同, 则该位的结果为0, 否则为1
 $\sim$: 取反, $\sim$ 是一元运算符, 用来对一个二进制数按位取反, 即将0变1, 将1变0
 $\ll$: 左移, 用来将一个数的二进制位全部左移N位, 右补0
 $\gg$: 右移, 将一个数的各二进制位右移N位, 移到右端的低位被舍弃, 对于无符号数, 高位补0。

位运算

■ 左移

在很多语言里左移的符号位 $\ll$,有些语言和伪代码常用shl

假设有个变量 $x=1$;

那么 $x=x\ll 1$ 表示的意思如下:

x : 000000000000000001 (二进制表示)

$x\ll 1$: 000000000000000010 (也就是把 x 的每一位向左移动1位的结果,右边不够的用0添上)

此时 $x=x\ll 1$,其实可以看做乘2

$x\ll i$ 表示的就是左移 i 位,可以看做乘 i 次2

位运算

■ 右移

在很多语言里右移的符号位 $\gg$, 有些语言和伪代码常用shr

假设有个变量 $x=2$;

那么 $x=x\gg 1$ 表示的意思如下:

x : 000000000000000010 (二进制表示)

$x\gg 1$: 000000000000000001 (也就是把 x 的每一位向右移动1位的结果, 左边不够的用0添上)

此时 $x=x\gg 1$, 其实可以看做除2

$x\gg i$ 表示的就是右移 i 位, 可以看做除 i 次2

位运算

■ 或运算

在很多语言里或运算的符号位 **|**，这个与逻辑运算符 **||** 是有区别的，而一些语言和伪代码也用 **or** 表示

假如有 $x=(5)_{10}=(101)_2$, $y=(2)_{10}=(010)_2$ (都是二进制表示)

那么 $x|y$ 表示如下

x: 101

y: 010

$x|y$: 111

其实也就是将 x 和 y 表示成二进制，然后按位做或运算

此时 $x|y==7$ (10进制表示)

位运算

■ 与运算

在很多语言里或运算的符号位 **&**，这个与逻辑运算符 **&&** 是有区别的，而一些语言和伪代码也用 **and** 表示

加上 $x=101$ $y=010$ (都是二进制表示)

那么 $x\&y$ 表示如下

$x:$ 101

$y:$ 010

$x\&y:$ 000

其实也就是将 x 和 y 表示成二进制，然后按位做与运算

此时 $x\&y==0$ (10进制表示)

位运算

■ 非运算(取反)

在很多语言里或运算的符号位 $\sim$ ，而一些语言和伪代码也用 not 表示

假设 $x=101$ (都是二进制表示)

那么 $\sim x$ 表示如下

$x:$ 101

$\sim x:$ 010

其实也就是将 x 表示成二进制，然后按位做取反运算

此时 $\sim x == 2$ (10进制表示)

位运算

■ 异或运算

在很多语言里或运算的符号位 $\wedge$ ，而一些语言和伪代码也用 xor 表示

加上 $x=1010$ $y=1100$ (都是二进制表示)

那么 $x \wedge y$ 表示如下

x: 1010

y: 1100

$x \wedge y$: 0110

其实也就是将 x 和 y 表示成二进制，然后按位做异或运算，也就是相同为0，不同为1

此时 $x \wedge y == 6$ (10进制表示)

位运算

■ 获取一个或多个固定位的值 (&)

假设 $x=1010$ (10进制的10)

我们要获取从右往左数第2位的值，那么我们可以这样来取

$x \& (1 \ll 1)$ 也就是

x: 1010

$1 \ll 1$: 0010

$x \& (1 \ll 1)$ 0010

这样我们就可以通过判断 $x \& (1 \ll 1)$ 是否等于0来知道这一位是0还是1了

当然我们可以用 $x \& (3 \ll 2)$ 来取得第3位和第4位
同时也可用 $x \& (x \ll 1)$ 来判断一个数相邻两位是不是同时为1

位运算

■ 把一个或多个固定位的值置零

假设 $x=1010$ (10进制的10)

我们把从右往左数第2位的值置零，那么我们可以这样来做

$x \& (\sim(1 \ll 1))$ 也就是

x : 1010

$\sim(1 \ll 1)$: 1101

$x \& (\sim(1 \ll 1))$ 1000

当然我们可以用 $x \& (\sim(3 \ll 2))$ 来把第3位和第4位置零

位运算

- ◆ 树状数组中取末尾连续多少个0的次方。
- ◆ 用 $x \& (-x)$ 表示。
- ◆ $X \& (-x)$ 等价于 $x \& ((\sim x) + 1)$
- ◆ 计算机中数字是以补码的形式进行存放的。
- ◆ 正整数的原码、反码和补码都一样；
- ◆ 负数部分：
 - ◆ 原码和反码的相互转换：符号位不变，数值位按位取反
 - ◆ 原码和补码的相互转换：符号位不变，数值位按位取反，末位再加1
 - ◆ 已知补码，求原码的负数的补码：符号位和数值位都取反，末位再加1

牛吃草

给出一个 n 行 m 列的草地 ($1 \leq M \leq 12; 1 \leq N \leq 12$), 1表示肥沃, 0表示贫瘠, 现在要把一些牛放在肥沃的草地上, 但是要求所有牛不能相邻, 问你有多少种放法 (mod 100000000)。

样例输入

2 3

1 1 1

0 1 0

样例输出

9

放牛的数量如下:

1 2 3

4

有四种方式放1头牛(1、2、3、4), 三种方式放两头牛(13、14、34), 1种方式放三头牛(134), 和1种1头牛都不放。

$4 + 3 + 1 + 1 = 9$ 。

分析

- ◇ 如何解决这个问题呢？即放牛的方式
- ◇ 1、问题中放1头牛的所有方法、~~2头牛的所有方法、3头牛的所有方法、....n头牛的所有方法？~~

2、一行一行的来放牛

分析

- ◆ 针对第1行放牛，则
 - ◆ 1、放牛的方法最多是从12个0到12个1。
 - ◆ 2、相邻两个位置不能都放牛（左右）。
 - ◆ 3、满足题目中该位置是肥沃的，否则也是不能放牛的。
- ◆ 针对第2行放牛，则
 - ◆ 1、放牛的方法最多是从12个0到12个1。
 - ◆ 2、相邻两个位置不能都放牛（左右）。
 - ◆ 3、还不能和第1行相同位置相邻（上线）
 - ◆ 4、满足题目中该位置是肥沃的，否则也是不能放牛的。

分析第 i 行

- ◆ 针对第 i 行放牛，则
- ◆ 1、放牛的方法最多是从12个0到12个1。
- ◆ 2、相邻两个位置不能都放牛（左右）。
- ◆ 3、还不能和第 $i-1$ 行相同位置相邻（上线）
- ◆ 4、满足题目中该位置是肥沃的，否则也是不能放牛的。

装压分析

- ◆ 由于每一行放牛中，如果我们按照每个位置存放起来，则比较相邻两格同时放牛是非常麻烦的。因此我们应该考虑，把这行看成一个由0和1组成的二进制数来完成。从而实现状态的压缩。
- ◆ 这样思考后，我们按照行进行放牛，就发现满足DP的过程。
- ◆ 所有可以用DP来解决这个问题。
- ◆ 我们定义： $dp[i][j]$:表示第*i*行状态为*j*的时候放牛的种数。
- ◆ *j*的范围0~~12个1即 $(2^{12}-1)$ 。这样子就可以把一行的所有情况表示完成。

分析3

- ◆ 状态压缩DP关键处理好位运算。
- ◆ 这里处理的是：
 - ◆ 1、一个二进制数相邻两位是不是同时为1（同行的判断）
 - ◆ 假如状态为 j 。 $j \& (j < < 1)$ 来实现
 - ◆ 2、两个二进制数相同位是不是同时为1（上下行的判断）
 - ◆ 假如第 $i-1$ 行状态为 x ,第 i 行状态为 y 。
 - ◆ $x \& y$ 来判断相同位是否为1

综合分析这个题目

- ◆ 针对每行DP，满足以下四点，则就是一种放置的方案。
- ◆ 1、放牛的方法最多是从12个0到12个1。
- ◆ 2、相邻两个位置不能都放牛（左右）。
- ◆ 3、还不能和第 $i-1$ 行相同位置相邻（上下）
- ◆ 4、满足题目中该位置是肥沃的，否则也是不能放牛的。
- ◆ 因此，针对第 i 行状态为 j 的方案数等于第 $i-1$ 行所有不与第 i 行
- ◆ 冲突的方案总数之和。

分析第三点和第四点有什么区别

- ◆ 第三点中，通过上下相同位置是否存在一个都为1的便不能满足要求。所有可以直接 $x \& y$ ，如果返回值为0这个表示一个都不存在，否则就是存在
- ◆ 第四点中。要求当前第 i 行的土地状态中某位是0的，则枚举的第 i 行的状态的同位置必须是0，否则不满足要求（牛只能放在肥沃的土地上）。如何来寻找呢？我们发现找0是比较困难的。所以可以把原来土地中是0的变成1，是1的变成0（即取反）以后再转换成第三点的情况。

◆ 那处理土地按位取反的操作呢？有两种方式

◆ 第一次：读取的时候就直接取0的位

◆ `for(int i=1;i<=n;i++)`

◆ `for(int j=1;j<=m;j++){`

◆ `scanf("%d",&x);`

◆ `if(x==0) map[i]+=(1<<(m-j));}`

◆ 第二种：正常读取，读完后在做&运算前，取个反

◆ `for(int i=1;i<=n;i++)`

◆ `for(int j=1;j<=m;j++){`

◆ `scanf("%d",&x);`

◆ `if(x==1) map[i]+=(1<<(m-j));}`

样例DP分析

◇ 1 1 1

◇ 0 1 0

◇ 第1行状态有:

◇ 000、001、010、100、101 (思考为什么?)

◇ 对应:

◇ $dp[1][0]=1$ 、 $dp[1][1]=1$ 、 $dp[1][2]=1$ 、
 $dp[1][4]=1$ 、 $dp[1][5]=1$

◇ 第2行状态有:

◇ 000、010 (思考为什么?)

◇ 对应: $dp[2][0]=5$ 、 $dp[2][2]=4$

◇ 所有合计有9种方案

实现代码：

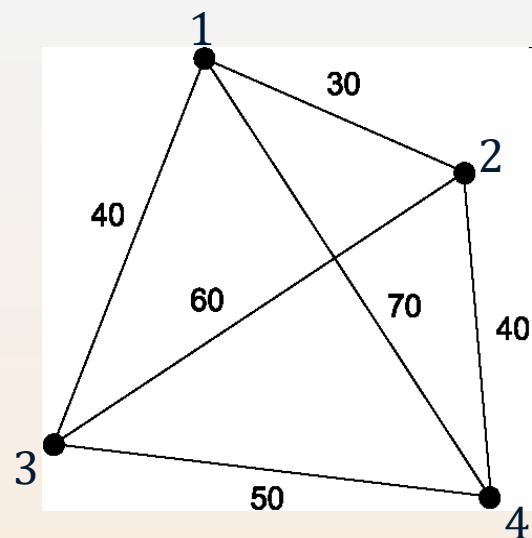
◆ 同学们可以自己完成了吗？



状态压缩动态规划

◆ 一道经典的题目---旅行商问题

- ◆ 描述：旅行商要到若干个城市旅行，各城市之间的费用是已知的，为了节省费用，旅行商决定从所在城市出发，到每个城市旅行一次后返回初始城市，问他应选择什么样的路线才能使所走的总费用最短？（ $N \leq 20$ ）



从1出发

最优路径 1 -> 2 -> 4 -> 3

状态压缩动态规划

◆ 暴力搜索

- ◆ 始点固定，枚举所有路径。
- ◆ 复杂度 $O(n!)$ 当 n 大于 10 --- TEL!!!

◆ 动态规划方法

- ◆ 状态定义： $dp[i]$ 表示：到达结点 i 点的最小花费。
- ◆ 正确么？
- ◆ 每个结点访问刚好一次 --- 不满足无后效性

状态压缩动态规划

- ◆ $dp[s][v]$
- ◆ 表示走完了点集 S 后，最后停留在 v 点的最小花费。
- ◆ 递推方程：
- ◆ $dp[s][v] = \min(dp[(s \text{ 除去点 } v)][k] + \text{dis}[k][v], dp[s][v])$

核心参考代码

```
for(int S = 0; S <= (1<<n)-1; ++S){ //枚举所有状态, 用位运算表示
    for(int i = 1; i <= n; ++i)
    {
        if(S & (1<<(i-1)))
            //状态S中已经经过城市i
        {
            if(S == (1<<(i-1))) dp[S][i] = dis[0][i];
            //状态S只经过城市i, 最优解自然是从0出发到i的dis, 这也是DP的边界
            else //如果S有经过多个城市
            {
                dp[S][i] = INF;
                for(int j = 1; j <= n; ++j)
                {
                    if(S & (1<<(j-1)) && j != i) //枚举不是城市i的其他城市
                        dp[S][i] = min(dp[S^(1<<(i-1))][j] + dis[j][i], dp[S][i]);
                    //在没经过城市i的状态中, 寻找合适的中间点j使得距离更短, 和FLOYD一样
                }
            }
        }
    }
}
```

noip2017 Day2 T2(宝藏)

- ◆ 题目大意：
- ◆ 给出一张无向图,每条边都有一个权值且均未开通,先可以随便取一个起点,要开通一些边,使它成为一个连通图,开通一条边的代价为权值*起点到它的点的个数(起点也算),求最小代价。
- ◆ 数据范围： $1 \leq n \leq 12$ ； $0 \leq m \leq 1000$ ； $v \leq 500000$ 。

宝藏

- ◆ 问题分析：这个题和旅行商问题很相似，就是选取起点和路径不一样，所产生的代价就不一样， $n \leq 12$ ，所以处理方法可以状态压缩DP来实现。

$X \& (x-1)$ 的作用：

将x的二进制值中，最后一个1置0，其它不变

方法一：记忆化搜索实现

- ◆ 用 $f[S]$ 表示已经取的节点的状态为 S 的最小代价，那么目标状态就是 $f[(1 \ll n) - 1]$ 。
- ◆ 如果考虑递推求解的话，我们发现需要计算花费的深度会很难枚举，这就导致了DP顺序的问题，如果选择用记忆化搜索求解的话，会方便很多。
- ◆ 由于起点不是确定的，我们需要用一重循环来枚举起点，然后以 $1 \ll (\text{root} - 1)$ 为初始状态，对每一种情况进行一次记忆化搜索

记搜思路

- ◆ 记搜的大体思路如下：
- ◆ 1.对于状态 S ，枚举一个点 i ，满足 $i \in S$
- 2.再枚举一个点 j ，满足 i, j 有边相连且 $j \notin S$
- 3.状态转移方程： $f[S'] = \min(f[S] + \text{depth}[i] * \text{dis}[i][j])$
- 4.更新新加入的节点 j 的深度
- 5.搜索下一个状态 S'
- 6.回溯还原节点 j 的深度
- ◆ 最后在每一种情况的目标状态中取个 $\min$ 就是最终的答案了，时间复杂度 $O(n^3 2^n)$ 。

记忆化搜索参考代码

```
void dfs(int S)
```

```
{
```

```
    for(int i=1;i<=n;i++)
```

```
    {
```

```
        if((1<<(i-1))&S)
```

```
        {
```

```
            for(int j=1;j<=n;j++)
```

```
            {
```

```
                if(not ((1<<(j-1))&S)&&(dis[i][j]<INF))
```

```
                {
```

```
                    if(f[S|(1<<(j-1))]>f[S]+depth[i]*dis[i][j])
```

```
                    {
```

```
                        int temp=depth[j];
```

```
                        depth[j]=depth[i]+1;
```

```
                        f[S|(1<<(j-1))]=f[S]+depth[i]*dis[i][j];
```

```
                        dfs(S|(1<<(j-1)));
```

```
                        depth[j]=temp;
```

```
                    }
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
void solve()
```

```
{
```

```
    for(int root=1;root<=n;root++)
```

```
    {
```

```
        memset(f,0x3f,sizeof f);
```

```
        memset(depth,0x3f,sizeof depth);
```

```
        f[1<<(root-1)]=0;
```

```
        depth[root]=1;
```

```
        Search(1<<(root-1));
```

```
        ans=min(ans,f[(1<<n)-1]);
```

```
    }
```

方法二：递推

- ◆ 通过记忆化搜索发现，这里用一维 $dp[i]$ 表示时，不好枚举，也就是会导致有后效性，前面我说，当不行时，**那么我们为了使dp无后效性可以加一维甚至多维来限制，达到无后效性的效果。**考虑到这个题的特征，最终的答案是一棵生成树，所以我们**增加深度这一维来维护dp特征。**
- ◆ 用 $dp[i][s]$ 表当前深度为 i ,状态为 j 的最小花费,则:
- ◆ $dp[i][s] = dp[i-1][t] + g[s][t] * (i-1)$ 。
- ◆ $g[s][t]$ 表示从**状态s**挖到**状态t**的最小花费。
- ◆ 时间复杂度 $O(3^n * n)$
- ◆ 为了预处理出 $g[t][s]$ ，我们可以再开一个 $h[i][s]$ 表示从点 i 挖到状态 s 的最小花费。
- ◆ 预算我们就可以用边权更新 h 数组，用 h 数组更新 g 数组

方法二递推

- ◇ g数组的预处理
- ◇ $g[i][j]$ 表示从i状态挖到j状态的最小花费

```
for (int s=1; s<ms; s++){
    int t=s&(s-1);
    while (t){
        int x=s^t;
        for (int i=1; i<=n; i++)
            if ( x&(1<<(i-1)) )
            {
                g[s][t]+=h[i][t];
                if (g[s][t]>oo) g[s][t]=oo;
            }
        t=s&(t-1);
    }
}
```

Dp过程

```
for (int i=1; i<=n; i++)
    for (int s=0; s<ms; s++) f[i][s]=oo;
for (int i=1; i<=n; i++) f[1][1<<(i-1)]=0;
for (int i=2; i<=n; i++)
    for (int s=1; s<ms; s++)
    {
        int t=s&(s-1);
        while (t)
        {
            int temp;
            if (g[s][t]<oo) temp=g[s][t]*(i-1);
            else temp=oo;
            if (f[i-1][t]<oo) f[i][s]=min(f[i][s],f[i-1][t]+temp);
            t=s&(t-1);
        }
    }

int ans=oo;
for (int i=1; i<=n; i++) ans=min(ans,f[i][ms-1]);
printf("%d\n",ans);
```

题解博客: <https://blog.csdn.net/a1035719430/article/details/80488083>

状态压缩动态规划练习题

- ◇ <https://www.cnblogs.com/ibilllee/p/7651971.html>
- ◇ <https://www.cnblogs.com/Tony-Double-Sky/p/9283254.html>
- ◇ <https://www.cnblogs.com/ibilllee/p/7651971.html>
- ◇ 状态压缩DP学习心得：
- ◇ <https://tbr-blog.blog.luogu.org/zhuang-ya-dp-chang-yong-cao-zuo>

数位dp

- ◆ 数位统计类DP是与数字相关的一类计数问题。在这类题目中，一般给定一些限制条件，求满足**限制条件的第K小的数是多少，或者求在区间[L,R]内有多少个满足条件的限制**。解决方法一般先用动态规划进行预处理，再基于拼凑思想，用“试填法”求出最终答案。
- ◆ 数位dp的实质就是换一种暴力枚举的方式，使得新的枚举方式(按位枚举)满足dp的性质，然后记忆化就可以了。

数位dp

- ◇ 例：
- ◇ 求所有n位数中能被m整除的数的个数。
- ◇ $N \leq 10^{15}$
- ◇ 问题分析：
- ◇ 朴素方法：从最小的n位数枚举到最大的n位数，然后判断是否能被m整除即可。
- ◇ 时间复杂度 $O(n)$ ，**超时，不解释！** 怎么办？
- ◇ 寻求新的解决方法，转变思路，直接枚举数字大小是不行的，那我按照n数字的位数，一位一位枚举，即数位DP。

问题分析

- ◆ 一般数位DP是多维的，首先会有一维表示的是当前到了第几位，通常情况这一维可以使用滚动。其它的就是根据题目的实际要求了，如例题就需要新增一维来记录除以 m 的余数。
- ◆ 以例题为例，我们来讲讲数位DP怎么实现。

状态与转移

- ◆ 设 $dp[i][j]$ 表示当前到了第 i 位，除以 m 的余数为 j 的方案数。
- ◆ 一般是从高位往低位DP，因为最高位不能为0，所以我们对 $i=1$ 的特殊情况预先处理好： $dp[1][j \bmod m]=1 (1 \leq j \leq 9)$
- ◆ 接着我们用第 i 位的状态去更新第 $i+1$ 位的状态，每次枚举当前在第 $i+1$ 位加入 k 。想想怎么转移？
- ◆ 对于当前的余数 j ，在末尾加入了 k ，余数变成什么？
- ◆ 根据同余，我们可以得到：余数变成了 $(j*10+k) \bmod m$ 。
- ◆ 所以，状态转移方程如下：
- ◆ $dp[i+1][(j*10+k) \bmod m] += dp[i][j] (1 \leq i < n, 0 \leq j < m, 0 \leq k \leq 9)$
- ◆ 最后的答案是什么？
- ◆ 首先，第一维自然是 n ，
- ◆ 那么第二维取什么值呢？
- ◆ 显而易见，要求是 m 的倍数，所以第二维是0。
- ◆ 综上所述，最后的答案是 $dp[n][0]$ 。

hdu:2089

- ◆ 题目大意:
- ◆ 从x到y求不包含62和4的数的个数。
- ◆ $0 < x \leq y < 1000000$
- ◆ 问题分析: 直接枚举, 肯定超时, 不解释
- ◆ 我们考虑求出0~y的个数, 再减去0~(x-1)的个数, 即为答案。
- ◆ $dp[i][j]$: 表示i位数, 最高位是j的数, 符合题意的数有多少个。
- ◆ 比如 $dp[1][2]=1$; $dp[1][4]=0$;
- ◆ $dp[2][6]=8$ (60,61,63,64,65,66,67,68,69)
- ◆ 我们先不关注这个dp有什么用, 我们先关注dp本身怎么求。
- ◆ 首先 $dp[1][i]=0$ (if $i==4$), $dp[1][i]=1$ (if $i \neq 4$) ($0 \leq i \leq 9$)。这一步是很显然的, 那么根据这个题的数据范围, 只需要递推到 $dp[7][i]$ 就够用了。

◆ 那么稍微理解一下，可以想出递推式：

◆ $dp[i][j]=$

if ($j==4$) $dp[i][j]=0$

else if ($j!=6$) $dp[i][j]=\Sigma(dp[i-1][k])$ ($k=0,1,2,3,4,5,6,7,8,9$)

else if ($j==6$) $dp[i][j]=\Sigma(dp[i-1][k])$ ($k=0,1,3,4,5,6,7,8,9$)

◆

```
void getdp()
{
    dp[0][0]=1;
    for(int i=1;i<10;i++){
        for(int j=0;j<10;j++){
            if(j==4) dp[i][j]=0;
            else if(j==6)
            {
                for(int k=0;k<10;k++) dp[i][j]+=dp[i-1][k];
                dp[i][j]-=dp[i-1][2];
            }
            else
            {
                for (int k=0;k<10;k++) dp[i][j]+=dp[i-1][k];
            }
        }
    }
}
```

◆

这样想：

肯定都不

果j不是6，

是dp[i-

是2就行，

为0的符合

。

- ◆ 那么现在我们已经得到了dp数组，再重申一下它的含义： i 位数，最高位是 j 的数，符合题意的数有多少个。
- ◆ 现在我们就关注怎么利用dp数组做出上面我们说的出 $[0,x]$ 中符合题意的数有多少个。
- ◆ 那么我们做这样一个函数`int solve(int x)` 它可以返回 $[0,x]$ 中符合题意的有多少个。
- ◆ 那么`solve(x+1)`实际上与 $[0,x]$ 是等价的。
- ◆ 那么现在问题转化成了：小于 x ，符合题意的数有多少个？
- ◆ 很简单，既然小于，从最高位开始比，必定有一位要严格小于 x (前面的都相等)。所以我们就枚举哪一位严格小于(前面的都相等)。

- ◆ 假设我们现在把 x 分成了 $a_1, a_2, \dots, a_L$ 这样一个数组，长度为 L ， a_L 是最高位。
- ◆ 那么结果实际上就是这样：
- ◆ 长度为 L ，最高位取 $[0, a_L - 1]$ 的所有的符合题意数的和；再加上长度为 $L - 1$ ，最高位取 a_L ，次高位取 $[0, a_{L-1} - 1]$ 的所有符合题意数的和；再加上……；一直到第一位。
- ◆ 我们还需要考虑这种情况：最高位 a_L 如果是4，那么这句话直接就可以终止了，因为粗体这句话前面的那句话“最高位取 a_L ”是不能成立的。还要考虑这种情况：最高位 a_L 如果是6，那么这里并不是能取 $[0, a_{L-1} - 1]$ 的所有（不能取2）。加上这些条件之后就严谨了。
- ◆ 上面的汉字对应到题目里，就是我们前面求出来的 $f[L][0..a_L - 1], f[L - 1][0..a_{L-1} - 1], \dots$

练习题目

- ◆ hdu:3555: 给定x, 输出从0~x所有的数中, 含有49的所有数的个数。
- ◆ hdu:A serious math problem
- ◆ 给出一个整数x, 求从0~x, 可以被13整除, 并且包含13 的数的个数。
- ◆ hdu:F(X)
- ◆ 给定两个数a,b。还有一个加权函数,
- ◆ $F(x) = A_n * 2^{n-1} + A_{n-1} * 2^{n-2} + \dots + A_2 * 2 + A_1 * 1$ 。
- ◆ 然后求, 0~b的各个数中进行F(x)不大于F(a)的数字的个数。

启示录(POJ 3208)

- ◆ 题目大意:
- ◆ 定义十进制下有3个连续的6的数为魔鬼数。有T个询问，求第X小的魔鬼数。
- ◆ $X \leq 5 \times 10^7$, 每个测试点包含不超过1000组数据。
- ◆ 问题分析:
 - ◆ 1、朴素方法，就是从最小的三位数开始枚举，然后判断该数是否是魔鬼数。最后找到第x小的魔鬼数。时间复杂度为 $O(x \times 8^3 \times 1000)$ 。肯定是不行的。

启示录(POJ 3208)

- ◆ 用动态规划的思想来做，我们先确定第 x 小的魔鬼数的位数，确定位数后，在用“试填”的方法，从左往右依次考虑每个数位。同时记录当前末尾已经有连续的几个6。
- ◆ 首先思考如何确定第 x 小的魔鬼数的位数，直接做是没法做的，我们只能找出 i 位数字有多少个魔鬼数来枚举处理。

启示录(P0J 3208)

- ◆ 用 $F[i][3]$ 表示由 i 位数字构成的魔鬼数有多少个。
- ◆ $F[i][j](0 \leq j \leq 2)$ 表示由 i 位数字构成且开头已经连续有 j 个6的非魔鬼数有多少个。
- ◆ 注意，在计算 F 时，我们允许前导0的存在。
- ◆ 考虑第 i 位(最高位)是什么数字，容易得到递归转移方程：
- ◆ $F[i][0] = 9 * (F[i-1][0] + F[i-1][1] + F[i-1][2])$
- ◆ $F[i][1] = F[i-1][0]$ $F[i][2] = F[i][1]$
- ◆ $F[i][3] = F[i-1][2] + 10 * F[i-1][3]$

```
void Init(){
    f[0][0]=1;
    for(int i=0;i<=20;i++){
        for(int j=0;j<3;j++){
            f[i+1][j+1]+=f[i][j];
            f[i+1][0]+=f[i][j]*9;
        }
        f[i+1][3]+=f[i][3]*10;
    }
}
```


- ◆ 通过 $F[i][3]$ 确定第 X 小的魔鬼数的位数，然后在用“试填”的方法，从左往右依次考虑每个数位。同时记录当前末尾已经有连续的几个6。
- ◆ 从小到大枚举当前位数填什么数字，通过预处理的 F 数组可以直接计算出后边的数位有多少种填法可以得到魔鬼数，与 X 比较即可，代码实现很具技巧性。

dp参考代码

```
while(f[m][3]<n) m++;
for(int i=m,k=0;i>0;i--){//试填第i位, 末尾已有K个6(k==3也表示已经是魔鬼数)
    for(int j=0;j<=9;j++){//从小到大枚举第i位填的数字
        long long cnt=f[i-1][3];//3个连续6全部来自于后i-1位
        if(j==6||k==3)//枚举的当前位是6或者已经有3个连续的6了
            for(int L=max(3-k-(j==6),0);L<3;L++)//妙用
                cnt+=f[i-1][L];
        if(cnt<n) n-=cnt;//如果cnt比n小, 说明第n个魔鬼数的第i位应该比j更大
        else//否则, 第i位就应该是j
        {
            if(k<3)
            {
                if(j==6) k++; else k=0;
            }
            printf("%d",j);
            break;
        }
    }
}
```

月之谜(BZOJ 1799)

- ◆ 题目大意:
- ◆ 如果一个十进制数能够被它的各位数字之和整除, 则称这个数为“月之数”。给定整数L和R, 你需要计算闭区间[L, R]中有多少个月之数。
- ◆ $1 \leq L, R \leq 2^{31}$, 每个测试点包含不超过3000组数据。